

LSPSS: Constructing Lightweight and Secure Scheme for Private Data Storage and Sharing in Aerial Computing

Haoyang Wang¹, Kai Fan¹, *Member, IEEE*, Chong Yu², *Graduate Student Member, IEEE*,
Kuan Zhang³, *Member, IEEE*, Fenghua Li, *Member, IEEE*, Hui Li⁴, *Member, IEEE*,
Yintang Yang⁵, *Senior Member, IEEE*, and Haojin Zhu⁶, *Fellow, IEEE*

Abstract—Aerial computing is gradually playing an essential role in edge and fog computing paradigms by virtue of mobility, availability, scalability, flexibility, and simultaneity, where the Low-altitude Computing (LAC) platform, as the end close to the data sources, is mainly responsible for data collection and storage. However, because of the long physical distance of data transmission and the vulnerability of the transmission link to various attacks, how to efficiently share the stored data while ensuring data privacy is a critical issue for LAC at present. In this article, we propose a lightweight and secure private data storage and sharing scheme to support range queries over encrypted multi-dimensional data. Specifically, we first propose two data conversion methods for transforming location features and collected log files with multi-dimensional attributes in Unmanned Aerial Vehicles (UAVs). Based

on the ideas of asymmetric scalar-product-preserving encryption (ASPE) and inner product comparison (IPC), we design a privacy-preserving storage and sharing technique for the converted data. In addition, to achieve secure and efficient data querying and result verification, we design a secure data index and build a data authentication structure (DAS) with G-tree. Finally, we rigorously analyze the security of our proposed scheme and conduct extensive experiments on a real-world database to prove that our proposed scheme is secure and easy to use in practical application scenarios.

Index Terms—Aerial computing, unmanned aerial vehicle (UAV), multi-dimensional data, privacy-preserving.

I. INTRODUCTION

AS RESEARCHERS defined the road-maps for sixth-generation (6G) networks, various Internet of Things (IoT) applications have more data to be collected and transmitted in 6G. To overcome the limitations of IoT devices in energy supply, transmission power and processing capability, and to facilitate data storage and sharing among devices in diverse regions, Aerial Access Networks (AANs) has been proposed to provide favorable channels and improved coverage. In addition, to be more applicable to computation-intensive applications such as autonomous vehicles, virtual reality (VR), and holographic communications, the edge computing has also been introduced for data collection and management in IoT devices. As a combination of AANs and edge computing, the emerging *aerial computing* is expected to improve the quality of services such as communication, computing, storage, sensing, navigation, and control on a global scale.

The typical aerial computing framework under the 6G architecture is shown in Fig. 1, where the aerial components are deployed in the aerial domain with vertical layering, i.e., (i) *IoT*; (ii) *Terrestrial Computing*; (iii) *Low-altitude Computing (LAC) Platform*; (iv) *High-altitude Computing (HAC) Platform*; and (v) *Satellite Computing Platform*. The IoT layer consists of any data-generating IoT devices, such as computation-intensive applications that require participation by aerial computing. The Terrestrial Computing layer contains traditional computing models such as fog computing, multi-access edge computing (MEC), and Cloudlet. The LAC, HAC, and satellite computing platforms provide storage and computing services at different

Manuscript received 3 May 2023; revised 4 October 2023; accepted 11 November 2023. Date of publication 16 November 2023; date of current version 9 October 2024. This work was supported in part by the “Pioneer” and “Leading Goose” R&D Program of Zhejiang under Grant 2022C03174, in part by the National Natural Science Foundation of China under Grants 62372356 and 92067103, in part by the Key Research and Development Program of Shaanxi under Grant 2021ZDLGY06-02, in part by the Henan Key Laboratory of Network Cryptography Technology under Grant LNCT2022-A13, in part by the Natural Science Foundation of Shaanxi Province under Grant 2019ZDLGY1202, in part by the Shaanxi Innovation Team Project under Grant 2018TD-007, in part by the Xi’an Science and Technology Innovation Plan under Grant 2018168CX9JC10, in part by the Fundamental Research Funds for the Central Universities under Grant YJS2212, and in part by the National 111 Program of China under Grant B16037. (Corresponding author: Kai Fan.)

Haoyang Wang and Kai Fan are with the State Key Laboratory of Integrated Services Networks, School of Cyber Engineering, Xidian University, Xi’an 710126, China, and also with the Henan Key Laboratory of Network Cryptography Technology, Zheng Zhou 450003, China (e-mail: why19970701@163.com; kfan@mail.xidian.edu.cn).

Hui Li is with the State Key Laboratory of Integrated Services Networks, School of Cyber Engineering, Xidian University, Xi’an 710126, China (e-mail: lihui@mail.xidian.edu.cn).

Chong Yu is with the Department of Computer Science, University of Cincinnati, OH 45221 USA (e-mail: yuc5@ucmail.uc.edu).

Kuan Zhang is with the Department of Electrical and Computer Engineering, University of Nebraska-Lincoln, Lincoln, NE 68588 USA (e-mail: kuan.zhang@unl.edu).

Fenghua Li is with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100093, China, and also with the State Key Laboratory of Integrated Services Networks, School of Cyber Engineering, Xidian University, Xi’an 710071, China (e-mail: lifenghua@iie.ac.cn).

Yintang Yang is with the Key Laboratory of the Ministry of Education for Wide Band-Gap Semiconductor Materials and Devices, Xidian University, Xi’an 710071, China (e-mail: ytyang@xidian.edu.cn).

Haojin Zhu is with the Department of Computer Science & Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: zhu-hj@cs.sjtu.edu.cn).

Digital Object Identifier 10.1109/TSC.2023.3333347

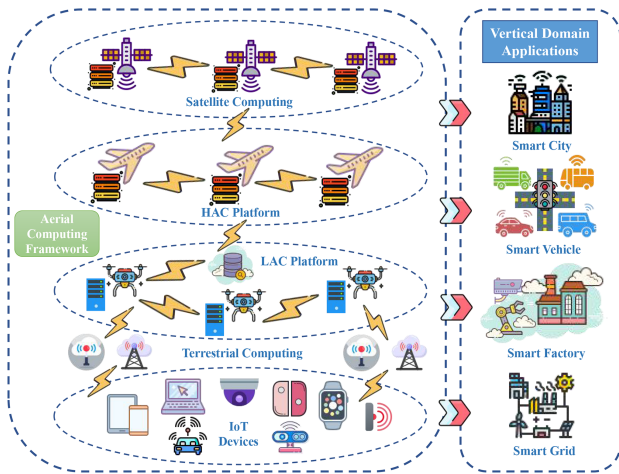


Fig. 1. Aerial computing framework under the 6G architecture.

altitudes for UAVs, manned vehicles, and LEO satellites, respectively. With the above framework, aerial computing not only retains the characteristics of edge computing such as on-premises, proximity, low latency, location awareness, and network contextual information, but also offers mobility, availability, scalability, flexibility, and simultaneity. These characteristics enable aerial computing to perform essential roles in smart cities, smart transportation, smart factories, and smart grids.

Although aerial computing brings many benefits, it still suffers from several security threats below. (i) The risk of privacy exposure during communication increases due to the long physical distance among components in the aerial domain. (ii) UAVs in the LAC platform often require centralized data collection to facilitate subsequent training, but sharing sensitive data in the aerial domain may leak private information. (iii) The each-kind component in the framework has different storage space and computation power, how to allocate these resources to support sensitive data storage and sharing is also an urgent issue to be solved.

In aerial computing, there exists some literature to preserve data privacy during data interactions with blockchain [1], [2], [3], cryptography [4], [5], and differential privacy [6]. However, for the blockchain-based scheme, (i) block-chain construction requires huge storage space and computation capacity; (ii) the data ledger only can store a few sensitive information; (iii) the operation of on-chain smart contracts brings extra overhead. For the cryptography-based scheme, the cryptographic primitives such as online/offline signatures and FHE cause huge computational and storage overhead, and cannot be applied to resource-limited wireless-aware devices. As for the differential privacy-based scheme, although this technique is less overhead (i.e., privacy enhancement is achieved by introducing artificial noise), it is likely to generate unusable results due to noise accumulation in complex computational tasks.

In summary, the existing privacy-preserving schemes have the following problems. (i) The security cannot meet the requirements of practical scenarios; (ii) The consumption of system construction is too large to be arranged on resource-constrained devices; (iii) The storage and sharing mode is single and cannot fulfill the user demands.

To address the limitations of these schemes, we design a lightweight and secure private data storage and sharing scheme called LSPSS in aerial computing, which aims to achieve the following objectives.

- 1) *Stronger Privacy Preservation*: The primary objective of storing and sharing private data in aerial computing is privacy preservation. Thus LSPSS requires achieving stronger privacy preservation compared to existing schemes. Specifically, it should enable semantic security under the *Known Background Model*, as well as the achievement of single-dimensional and query unlinkability.
- 2) *Multi-dimensional Data*: UAVs in aerial computing collect and process private data consisting of multi-dimensional attributes. Thus LSPSS needs to focus on the storage and sharing of multi-dimensional data.
- 3) *Lightweight Overhead*: Most existing schemes result in huge space and computation overhead for privacy preservation, thus reducing the required overhead while preserving data privacy is also a key objective that needs to be achieved in LSPSS.

Our Contribution: To achieve these objectives, we address three challenges. The first one is to convert the original multi-dimensional data into an easily stored and shared form. Inspired by the concept of IPC [7], we convert the multi-dimensional data into vectors that can be computed with IPC. Then, we implement privacy-preserving storage and sharing of the vectors based on the ASPE algorithm and cryptographic functions. Moreover, we design a verification mechanism for completeness based on the Merkle tree to enhance the reliability of query results. Our contributions are summarized as follows.

- Aiming at two types of multi-dimensional data (i.e., location and log files) of UAVs, we propose two conversion methods to convert multi-dimensional data into vectors. These approaches not only facilitate the storage, but also enables the sharing of multi-dimensional data.
- To achieve lightweight multi-dimensional data storage and sharing with privacy preservation, we first design two secure data indexes for multi-dimensional location and log files, respectively. Then we propose two permutation and perturbation-based matrix encryption methods to encrypt private data.
- In addition, we propose a verification mechanism for the completeness of query results by combining the data index and Merkle tree (e.g., it can require 400 milliseconds to complete the result verification over 25000 files).
- To validate the efficiency of LSPSS, we not only rigorously analyze its security but also conduct extensive experiments on real-world datasets (e.g., it can require 2.5 seconds to perform a query over 25000 files).

We organize the remainder of this paper as follows. In Section II, we introduce the related work. Then we depict some preliminaries about LSPSS in Section III. In Section IV, we present the system model, the security and algorithm definition of LSPSS. In Section V, we describe our LSPSS construction in detail, followed by security analysis and performance evaluation in Sections VI and VII, respectively. Finally, we draw conclusions in Section VIII.

II. RELATED WORK

As our proposed LSPSS focuses on the secure storage and sharing of multi-dimensional data, we introduce several typical schemes aimed at this issue as well.

To reserve of plaintext order in the ciphertexts, order-preservation encryption (OPE) is applied in most of the early range query schemes [8], [9]. However, due to the leakage of the plaintext order [10], OPE-based range query schemes cannot guarantee security under the ordered chosen plaintext attack.

After that, some literature [11], [12], [13], [14] proposed range query schemes with buckets. These works split the plaintext space into buckets, so the order information in the same bucket can be preserved. Though, the split of data space with buckets causes false positive in query results, so users need extra consumption to filter the returned results in most bucket-based range query schemes.

Compared with the above range query schemes, public key-based range query schemes [15], [16], [17] are introduced to enhance security. The work [15] divided one multi-dimensional range query (MRQ) into several single-dimensional range queries, thus the single-dimensional privacy¹ is disclosed to servers. Moreover, public key cryptography such as hidden vector encryption (HVE) employed in these works [16], [17] brings huge computation consumption, thus not applicable in practical scenarios.

Considering the balance between security and efficiency, the currently mainstream privacy-preserving MRQ schemes utilized tree structures to design data indexes for secure and efficient retrieval, where the works are based on KD-tree [18], [19], [20], [21], [22] and the works are based on R-tree [23], [24], [25], [26], [27], [28], [29], while the works and employed B+-tree [7], [30], AVL-tree [31] and Internal tree [32], [33], respectively. However, all the above are three-party system models, which are not applicable to the architecture of aerial computing, so as we set in our design objectives, LSPSS should provide the solution to the issue of secure and efficient storage and sharing of multi-dimensional data in aerial computing.

III. PRELIMINARIES

In this section, we introduce some preliminaries used in LSPSS, including APSE, the cryptographic hash function (SHA-256), and the permutation function. For simplicity, the details of the G-tree [34] and Merkle tree [35] adopted in LSPSS are not introduced here.

A. Asymmetric Scalar-Product-Preserving Encryption

The literature [36] introduced asymmetric scalar-product-preserving encryption (ASPE) to enable the scalar product computation of two encrypted vectors. For privacy preservation, the ASPE encrypts data vectors and query vectors in different ways. The specific construction is illustrated below.

- **ASPE.Setup**(d) $\rightarrow SK$: Given the security parameter λ , generate the secret key SK , consisting of two invertible

matrices $M_1, M_2 \in \mathbb{R}^{\lambda \times \lambda}$ and one bit string $S \in \{0, 1\}^\lambda$ as split vector.

- **ASPE.Data_Enc**(SK, P) $\rightarrow \hat{P}$: Given the secret key SK and a data vector $P \in \mathbb{R}^\lambda$, split P into two vectors $\{P_1, P_2\}$ with split vector S as:

$$\begin{cases} P_1[i] + P_2[i] = P[i](S[i] = 1, i \in [1, \lambda]) \\ P_1[i] = P_2[i] = P[i](S[i] = 0, i \in [1, \lambda]) \end{cases} \quad (1)$$

Generate the encrypted data vector $\hat{P} = \{\hat{P}_1 = M_1^T P_1, \hat{P}_2 = M_2^T P_2\}$.

- **ASPE.Query_Enc**(SK, Q) $\rightarrow \hat{Q}$: Given the secret key SK and a query vector $Q \in \mathbb{R}^\lambda$, split Q into two vectors $\{Q_1, Q_2\}$ with split vector S as:

$$\begin{cases} Q_1[i] + Q_2[i] = Q[i](S[i] = 0, i \in [1, \lambda]) \\ Q_1[i] = Q_2[i] = Q[i](S[i] = 1, i \in [1, \lambda]) \end{cases} \quad (2)$$

Generate the encrypted query vector $\hat{Q} = \{\hat{Q}_1 = M_1^{-1} Q_1, \hat{Q}_2 = M_2^{-1} Q_2\}$.

- **ASPE.Match**($\hat{P}, \hat{Q}$) $\rightarrow Score$: The scalar product of plaintexts P and Q can be calculated by encrypted data vector $\hat{P}$ and query vector $\hat{Q}$ as:

$$\begin{aligned} \hat{P} \cdot \hat{Q} &= \hat{P}_1 \cdot \hat{Q}_1 + \hat{P}_2 \cdot \hat{Q}_2 \\ &= M_1^T P_1 \cdot M_1^{-1} Q_1 + M_2^T P_2 \cdot M_2^{-1} Q_2 \\ &= P \cdot Q \end{aligned} \quad (3)$$

B. Cryptographic Functions

1) *Hash Function*: To facilitate the Merkle tree to achieve the verification of query results, we employ the cryptographic hash function SHA-256 $H(\cdot)$ and the secret key K_H to calculate the hash value of each data record:

$$Hash_D = H(D, K_H) \quad (4)$$

The function possesses two features such as (i) collision-resistance: for two different data records D_1, D_2 , it generates different hash values; (ii) one-way: it is impossible to recover the data record D from $H(D, K_H)$. To simplify the expression, we use $H(\cdot)$ to denote $H(\cdot, K_H)$.

2) *Permutation Function*: To construct the matrix encryption methods to encrypt vectors, we employ the cryptographic permutation function $\Omega(\cdot)$ and the secret key K_Ω to obtain new arrangements:

$$\Omega_X = \Omega(X, K_\Omega) \quad (5)$$

The function also possesses two features such as (i) rearrangement: for one set X , it generates a rearrangement Ω_X of X , not an arrangement itself; (ii) one-way: it is impossible to recover the set X from $\Omega(X, K_\Omega)$ without K_Ω . Simply, we use $\Omega(\cdot)$ to denote $\Omega(X, K_\Omega)$.

IV. PROBLEM FORMULATION

In this section, we briefly explain our proposed approach. Then, we present the formal definition of the LSPSS. Finally,

¹The details of single-dimensional privacy is presented in *Security Definition*.

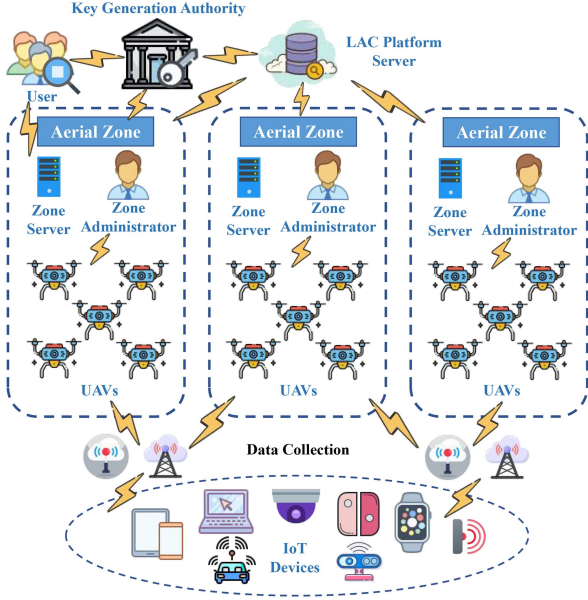


Fig. 2. System model of LSPSS.

we describe the leakage and give the security definition of our system.

A. System Model

The system model of LSPSS is shown as Fig. 2, which consists of six entities: (i) *UAVs*. UAVs have lower computation capacity and smaller storage space. UAV is responsible for collecting data and uploading them to zone servers and LAC platform after processing. (ii) *Zone servers (ZS)*. Each aerial zone possesses one ZS, which has certain computation power and storage space. The ZS primarily stores the location information of all UAVs in the zone, and responds to user queries for UAV locations. (iii) *Zone Administrators (ZA)*. Each aerial zone owns a unique administrator as a fully trusted entity, who builds the secure index and the DAS from the plaintext data (except location information), and later transfers the GeoHash code, index and DAS of the zone to the LAC platform together. (iv) *Key Generation Authority (KGA)*. The KGA acts as a fully trusted entity and generates keys and security parameters for the remaining entities in the system. (v) *Users*. Users can initiate queries with requirements, and decrypt the obtained query results after the completeness verification. (vi) *LAC platform server*. These servers have powerful computation capacity and storage space. They mainly store the log indexes and encrypted logs included in each zone, and respond to user queries.

B. Definition of LSPSS

Definition 1. (LSPSS): The LSPSS is a tuple Π that consists of five polynomial-time algorithms such as:

- $\{H(\cdot), \Omega(\cdot), SK\} \leftarrow \text{GenKey}(1^\lambda)$: It takes a security parameter λ as input, and outputs a secret key set SK , a permutation function $\Omega(\cdot)$ with its key K_Ω , a hash function $H(\cdot)$ with its key K_H .

- $\{I_L, I_D, E(\log)\} \leftarrow \text{GenIndex}(D, L, SK)$: It takes a secret key set SK , a multi-dimensional dataset D , a location dataset L as input, and outputs a secure data index I_D , a secure location index I_L , encrypted logs $E(\log)$.
- $\{TK_L, TK_D\} \leftarrow \text{GenToken}(Q_D, Q_L, SK)$: It takes a data MRQ Q_D , a location MRQ Q_L , a secret key set SK as input, and outputs a location token TK_L , a data token TK_D .
- $\{D_R, AVI\} \leftarrow \text{Search}(TK_L, TK_D, I_L, I_D)$: It takes two secure indexes I_L and I_D , two encrypted tokens TK_L and TK_D as input, and outputs query results D_R , auxiliary verification information AVI .
- $\{b \in \{0, 1\}, P_R\} \leftarrow \text{Verify}(D_R, AVI, SK)$: It takes query results D_R , auxiliary verification information AVI as input, then outputs a bit $b \in \{0, 1\}$ and the plaintext P_R of query results D_R .

C. Security Definition

As we discussed before, the LSPSS in this paper needs to achieve security under the known background model for semi-trusted adversaries. An adversary $\mathcal{A}$ can access three-type information, i.e., *History*, *View*, and *Trace*.

- *History*: An interaction among users and the server, consists of a dataset $\mathcal{D}$, a secure data index $\mathcal{I}$, and a query set $\mathcal{Q} = (q_1, \dots, q_\tau)$, uploaded by users, represented as the information $H = (\mathcal{D}, \mathcal{I}, \mathcal{Q})$.
- *View*: The server only can touch the encrypted form of H (i.e., the views) with the secret key SK , represented as $V(H)$. The views contains the encrypted dataset $\mathcal{D}^*$, the secure data index $\mathcal{I}^*$ and the query tokens $TK(\mathcal{Q})$.
- *Trace*: The trace $Tr(H)$ represents the information that the server can learn from a given history H . It consists of the access pattern $\alpha(H)$, search pattern $\sigma(H)$ and the returned identifiers $ID(\mathcal{Q})$. Let $ID(\Pi)$ represent the set of identifiers of the qualified data records, i.e., $\alpha(H) = \{ID(\Pi_1), \dots, ID(\Pi_\tau)\}$. The search pattern $\sigma(H)$ is described by a $n \times \tau$ binary matrix, where $\sigma(H)_{i,j}$ is 1 if $\mathcal{I}_i$ is matched to a query q_j , and 0 otherwise. Thus the views are denoted as $Tr(H) = \{ID(\mathcal{Q}), \alpha(H), \sigma(H)\}$.

Suppose that the server acquires the $Tr(H)$, several queries and their probability pairs (q_i, p_i) under the known background model. Under this attack model, we need to guarantee semantic security, which is defined as follows.

1) *Semantic Security*: The semantic security of the given encryption is modeled by a game played by a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$. The challenger generates an encryption scheme, while the adversary tries to break the scheme. the formal security model can be depicted as follows.

- *Setup*: Let SP be the system parameters. The $\mathcal{C}$ performs the key generation algorithm to generate a keypair (k_1, k_2) and sends to k_2 to the $\mathcal{A}$. The $\mathcal{C}$ keeps k_1 to respond to decryption queries from the $\mathcal{A}$.
- *Phase 1*: The $\mathcal{A}$ makes decryption queries on ciphertext that are adaptively chosen by the $\mathcal{A}$ itself. For a decryption query on the ciphertext CT_i , the $\mathcal{C}$ performs the decryption algorithm and then sends the plaintext to the $\mathcal{A}$.

- *Challenge*: The $\mathcal{A}$ outputs two distinct messages m_0, m_1 from the same message space, which are adaptively chosen by the $\mathcal{A}$ itself. The $\mathcal{C}$ randomly choose $c \in \{0, 1\}$ and then computes a challenge ciphertext $CT^* = E[SP, k_2, m_c]$, which is given to the $\mathcal{A}$.
- *Phase 2*: The $\mathcal{C}$ responds to decryption queries in the same way as in Phase 1 with the restriction that no decryption query is allowed on CT^* .
- *Guess*: The $\mathcal{A}$ outputs a guess c' of c and wins the game if $c' = c$. The advantage ε of the $\mathcal{A}$ in winning this game is defined as

$$\varepsilon = 2 \left(\Pr[c' = c] - \frac{1}{2} \right).$$

Definition 2. (Semantic Security): An encryption scheme is (t, ε) -secure in the security model of indistinguishability against chosen-plaintext attacks (IND-CPA) if there exists no adversary who can win the above game in time t with advantage ε , where the adversary is not allowed to make any decryption query.

Furthermore, single-dimensional privacy preservation and query unlinkability are also crucial security requirements during data sharing. We define both of them as follows.

2) *Single-Dimensional Privacy*: In the design objectives of LSPSS, privacy-preserving storage and sharing is oriented to multi-dimensional data, while *single-dimensional privacy*, as an essential security property in multi-dimensional data query, also belongs to the privacy-preserving requirements of LSPSS. The details of single-dimensional privacy have been provided by the literature [28], thus we introduce its formal definition directly.

Definition 3. (Single-Dimensional Privacy): Given a w -dimensional dataset $\mathcal{D} = \{ID_1, \dots, ID_n\}$ and a query $\mathcal{Q}$, the single-dimensional privacy of $\mathcal{D}$ refers to the information on which records satisfy each single dimension of the query, i.e., w record sets $Q_i = \{ID_x, ID_y, ID_z\} (i \in [1, w])$.

3) *Query Unlinkability*: The *query unlinkability* is an indispensable privacy-preserving requirement of the query over encrypted data. Briefly, if query tokens generated by the identical query are also the same, the servers can count the frequency of initiating diverse queries, and then infer part of query content with background knowledge. Thus, LSPSS also requires achieving query unlinkability. Its formal definition is given below.

Definition 4. (Query Unlinkability): Given a query $\mathcal{Q}$, and generate η query tokens $\{TK_1(\mathcal{Q}), \dots, TK_\eta(\mathcal{Q})\}$ for $\mathcal{Q}$, the query unlinkability can be achieved if all the m tokens are not the same.

4) *Completeness*: To improve the availability of query results, the users are required to perform completeness verification before decrypting the query results, to ensure that all records (logs) satisfying the query requirements are returned. Therefore, we present a formal definition of completeness as follows.

Definition 5. (Completeness): Given a query $\mathcal{Q}$, the server returns the result set $D_R = \{d_1, d_2, \dots, d_r\}$. If each record $f_i \in F$ such that f_i is in $\mathcal{Q}$, the encrypted data d_i must in D_R , then D_R is complete.

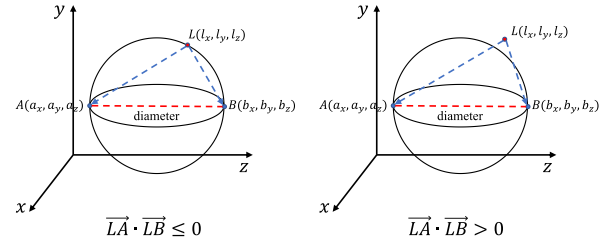


Fig. 3. Two cases for location.

V. TWO DATA CONVERSION METHODS FOR MULTI-DIMENSIONAL FEATURES

Generally speaking, the monitoring log files of UAVs are labeled by several feature attributes such as *temperature*, *humidity*, *concentration* and so on, and the location of UAVs are labeled by three attributes such as (*longitude*, *latitude*, *altitude*). In this section, aiming at facilitating the storage and sharing of multi-dimensional features, we introduce two data conversion methods, i.e., the conversion method for location data called CMLD and the conversion method for log files called CMLF below.

A. Conversion Method for Location Data

As Fig. 3 shows, given two points $A(a_x, a_y, a_z), B(b_x, b_y, b_z)$ in the zone, the sphere constructed with the line between A and B as the diameter is the query range S for the location of UAVs. Suppose that location of one UAV in the zone is $L(l_x, l_y, l_z)$, there exists the following conclusion (we consider that points on the boundary of the sphere are also located in the sphere). (i) $\vec{LA} \cdot \vec{LB} \leq 0$, then the UAV inside the S ; (ii) $\vec{LA} \cdot \vec{LB} > 0$, then the UAV outside the S .

Note that both vectors $\vec{LA}$ and $\vec{LB}$ include data information. To facilitate the storage and sharing of data, the location L and the query range S need to be divided into two independent vectors, thus we propose the following conversions of $\vec{LA} \cdot \vec{LB}$ based on the concept of IPC. In detail, $\vec{LA}$ and $\vec{LB}$ are defined as:

$$\begin{cases} \vec{LA} = (a_x - l_x, a_y - l_y, a_z - l_z) \\ \vec{LB} = (b_x - l_x, b_y - l_y, b_z - l_z) \end{cases} \quad (6)$$

Later, $\vec{LA} \cdot \vec{LB}$ can be divided into an expression of two independent vectors:

$$\begin{aligned} \vec{LA} \cdot \vec{LB} &= (a_x - l_x)(b_x - l_x) + (a_y - l_y)(b_y - l_y) + (a_z - l_z)(b_z - l_z) \\ &= (a_x b_x - a_x l_x - b_x l_x + l_x^2) + (a_y b_y - a_y l_y - b_y l_y + l_y^2) \\ &\quad + (a_z b_z - a_z l_z - b_z l_z + l_z^2) \\ &= [1 \cdot (a_x b_x) + l_x \cdot (-a_x) + l_x \cdot (-b_x) + 1 \cdot l_x^2] + [1 \cdot (a_y b_y) \\ &\quad + l_y \cdot (-a_y) + l_y \cdot (-b_y) + 1 \cdot l_y^2] + [1 \cdot (a_z b_z) + l_z \cdot (-a_z) \\ &\quad + l_z \cdot (-b_z) + 1 \cdot l_z^2] \\ &= (1, l_x, l_x, l_x^2, 1, l_y, l_y, l_y^2, 1, l_z, l_z, l_z^2) \end{aligned}$$

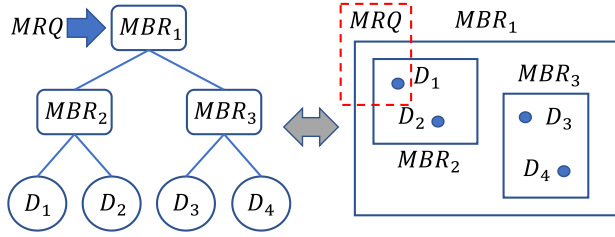


Fig. 4. Instance of G-tree and MRQ.

$$\begin{aligned} & \circ (a_x b_x, -a_x, -b_x, 1, a_y b_y, -a_y, -b_y, 1, a_z b_z, -a_z, -b_z, 1) \\ &= \sum_{i=1}^{12} u_i \cdot v_i. \end{aligned} \quad (7)$$

The correlation between $\vec{LA} \cdot \vec{LB}$ and $\sum_{i=1}^{12} u_i \cdot v_i$ can be formulated as:

$$\begin{cases} \vec{LA} \cdot \vec{LB} \leq 0 \Leftrightarrow \sum_{i=1}^{12} u_i \cdot v_i \leq 0 \\ \vec{LA} \cdot \vec{LB} > 0 \Leftrightarrow \sum_{i=1}^{12} u_i \cdot v_i > 0 \end{cases} \quad (8)$$

B. Conversion Method for Log Files

Similarly, the w -d attribute values of log files and the MRQ need to be divided into two-kind independent vectors. Also, to improve query efficiency and accuracy, we design the data index of log files based on the G-tree, where the non-leaf and leaf nodes correspond to w -d attribute ranges and values, respectively. As Fig. 4 describes, each non-leaf node represents one minimum bounding rectangle (MBR) and each leaf node represents one point, as well as the MRQ represents one hyper-rectangle. Thus, queries of log files can be converted into intersection tests between MRQ and nodes, as well as inclusion tests between MRQ and leaf nodes.

- 1) *Range Intersection*: Given a w -d data range $MBR = (R_1, \dots, R_w)$, a query $MRQ = (Q_1, \dots, Q_w)$, if the k -d attribute of MBR ($R_k = [l_k, h_k]$) and that of MRQ ($Q_k = [x_k^l, x_k^r]$) intersect, there exists a true proposition in (9) based on IPC. Furthermore, we add a perturbation δ (a random positive number) to this comparison predicate to acquire $\hat{p} = (l_k - x_k^l)(h_k - x_k^r)(h_k + \delta)$. Then, we express $\hat{p}$ as two independent vectors (i.e., query vector and range vector) in (10).

$$R_k \cap Q_k \neq \emptyset \Leftrightarrow p = (l_k - x_k^l)(h_k - x_k^r) \leq 0 \quad (9)$$

$$\vec{q} = \begin{pmatrix} x_k^r + \delta \\ -(x_k^r)^2 - x_k^r \delta \\ -x_k^l x_k^r - x_k^l \delta \\ x_k^l x_k^r (x_k^r + \delta) \end{pmatrix} \vec{v} = \begin{pmatrix} l_k h_k \\ h_k \\ l_k \\ 1 \end{pmatrix} \quad (10)$$

Obviously, whether the two ranges intersect or not is related to $\vec{q} \cdot \vec{v}$ as described in:

$$\begin{cases} \vec{q} \cdot \vec{v} \leq 0 \Leftrightarrow R_k \cap Q_k \neq \emptyset \\ \vec{q} \cdot \vec{v} > 0 \Leftrightarrow R_k \cap Q_k = \emptyset \end{cases} \quad (11)$$

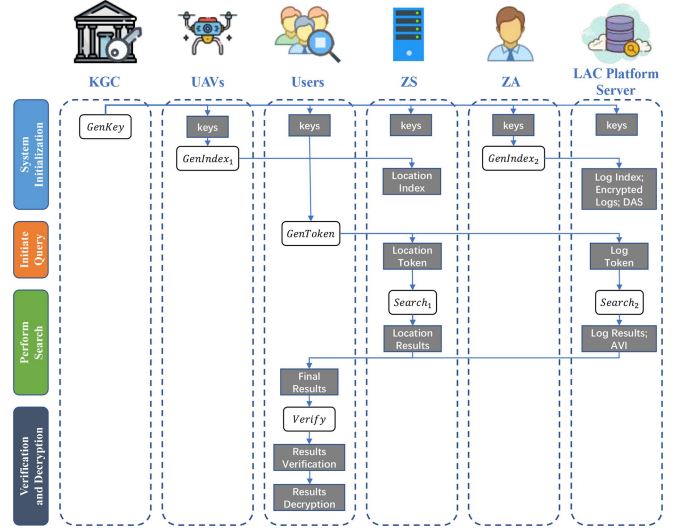


Fig. 5. System process of the LSPSS.

- 2) *Point Inclusion*: Given a w -d data point (i.e., data record) $D = (d_1, \dots, d_w)$, a query $MRQ = (Q_1, \dots, Q_w)$, if the k -th dimensional attribute value of D falls within the corresponding attribute range of MRQ , there exists a true proposition in (12) based on IPC. Also, we add a perturbation δ to this comparison predicate to acquire $\tilde{p}(d_k) = (d_k - x_k^l)(d_k - x_k^r)(d_k + \delta)$. Likewise, $\tilde{p}(d_k)$ is divided into one query vector and one value vector in (13).

$$x_k^l \leq d_k \leq x_k^r \Leftrightarrow \tilde{p}(d_k) = (d_k - x_k^l)(d_k - x_k^r) \leq 0 \quad (12)$$

$$\vec{q} = \begin{pmatrix} 1 \\ -x_k^l - x_k^r + \delta \\ x_k^l x_k^r - x_k^l \delta - x_k^r \delta \\ x_k^l x_k^r \delta \end{pmatrix} \vec{v} = \begin{pmatrix} d_k^3 \\ d_k^2 \\ d_k \\ 1 \end{pmatrix} \quad (13)$$

As shown in (14), whether the k -th attribute value of D falls within the corresponding attribute range of MRQ or not is related to $\vec{q} \cdot \vec{v}$.

$$\begin{cases} \vec{q} \cdot \vec{v} \leq 0 \Leftrightarrow d_k \in [x_k^l, x_k^r] \\ \vec{q} \cdot \vec{v} > 0 \Leftrightarrow d_k \notin [x_k^l, x_k^r] \end{cases} \quad (14)$$

Note that a D falls within the MRQ iff the value of each dimensional attribute of D falls within the corresponding attribute range of the MRQ . Similarly, a MBR intersects with the MRQ iff the range of each dimensional attribute of MBR intersects with the corresponding attribute range of the MRQ .

VI. LSPSS CONCRETE CONSTRUCTION

In the previous section, we elaborate on our LSPSS based on two methods CMLD and CMLF. First, we give the correlation among the corresponding algorithms of each entity in Fig. 5.

A. GenKey

The KGA performs this algorithm for generating keys and functions for other system entities. First, KGA generates two $(12 + N) \times (12 + N)$ -d invertible matrices M_1 , M_2 , a $(4 + M) \times (4 + M)$ -d invertible matrix M_3 and a $(12 + N)$ -d split vector S using the security parameter λ , and computes $\tilde{M}_1 = |\det(M_1)|M_1^{-1}$, $\tilde{M}_2 = |\det(M_2)|M_2^{-1}$, $\tilde{M}_3 = |\det(M_3)|M_3^{-1}$, where $|\det(M_1)|$, $|\det(M_2)|$ and $|\det(M_3)|$ are the absolute values of the determinant of M_1 , M_2 and M_3 .

KGA also gives a permutation function $\Omega(\cdot)$, a cryptographic hash function $H(\cdot)$ with their keys K_Ω , K_H . Then, KGA generates the symmetric key k of the AES algorithm and the keypair (pk, sk) of the RSA algorithm. The KGA distributes the keys and the functions to the respective entities.

B. GenIndex

We store the location data and the log files on the ZSSs and LAC platform, respectively. Since the two kinds of data correspond to diverse vectors after feature conversion, we build diverse data indexes.

- *Stage 1: Location Index Generation*

As designed in CMLD, the UAV first generates the plaintext of the location vector L based on the location data (l_x, l_y, l_z) and adds N 1s to L to obtain L' , as shown in (15). Then UAV obfuscates vector L' as L'' with permutation function $\Omega(\cdot)$ and key K_Ω . Then, the UAV splits L'' to obtain two $(12 + N)$ -d vectors with the split vector S , as shown in (16).

$$\begin{aligned}
 L &= (1, l_x, l_x, l_x^2, 1, l_y, l_y, l_y^2, 1, l_z, l_z, l_z^2) \\
 &\Downarrow \\
 L' &= (1, l_x, l_x, l_x^2, 1, l_y, l_y, l_y^2, 1, l_z, l_z, l_z^2, \underbrace{1, \dots, 1}_N) \\
 &\Downarrow \\
 L'' &= (\underbrace{1, 1, l_x, l_y, 1, l_z, 1^2, \dots, 1, 1, 1, l_z, l_x^2, l_y^2}_{12+N})
 \end{aligned} \tag{15}$$

$$\begin{cases} L_1[i] + L_2[i] = L''[i](S[i] = 1, i \in [1, 12 + N]) \\ L_1[i] = L_2[i] = L''[i](S[i] = 0, i \in [1, 12 + N]) \end{cases} \tag{16}$$

Later, the UAV employs two keys $\tilde{M}_1$, $\tilde{M}_2$ to encrypt the split vector $L'' = \{L_1, L_2\}$, and uploads the ciphertexts $C_L = \{L_1\tilde{M}_1, L_2\tilde{M}_2\}$ and $E(U_i)$ to the ZS, where $E(U_i)$ denotes as the AES ciphertext of UAV identifier.

- *Stage 2: Log Index and DAS Generation*

As designed in CMLF, all UAVs located in one aerial zone encrypt the log files (labeled by UAVs' identifiers) and multi-dimensional attribute vector sets (considered as multi-dimensional data points) with the AES algorithm and upload them to the ZA.

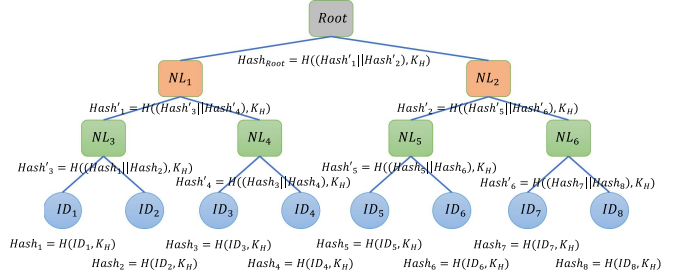


Fig. 6. DAS construction.

After receiving the encrypted data from all UAVs and decrypting the data points, the ZA first builds the plaintexts of the log index with the G-tree. Each leaf node in the tree contains a data point and a log file pointer, while each non-leaf node contains a data range covering all the child nodes.

Moreover, the ZA applies the hash function $H(\cdot)$ and the key K_H to compute the hash value $H(ID_i, K_H)$, where $ID_i (i \in [1, n])$ donates the identifier of each data record. As shown in Fig. 6, the ZA links the hash values of leaf nodes covered by the same parent node with the concatenation operation, and also computes the hash value of this linked string as the non-leaf node hash $Hash(NL_j) (j \in [1, m])$. The operation is repeated until the hash value of the root node $Hash(Root)$ is computed. The ZA encrypts the hash value of each node with the RSA private key sk , all signatures are stored on the corresponding node.

$$\begin{cases} Sig(Hash(ID_i)) (i \in [1, n]) \\ Sig(Hash(NL_j)) (j \in [1, m]) \end{cases} \tag{17}$$

For the plaintexts of multi-dimensional data points and ranges in the index, the ZA performs the following encryption operations. (i) As described in (18), the ZA inserts w random 4-d vectors into the w attribute vectors (corresponds to a w -d data range included in a non-leaf node), where all random numbers $r_k \ll x_{k,l}$ and $r_k \ll x_{k,r}$, ($k \in [1, w]$). Later, M 1s are added to each vector $RV_j (j \in [1, 2w])$, then each RV_j is obfuscated by the $\Omega(\cdot)$ and K_Ω . The ZA encrypts each obfuscated vector with the key $\tilde{M}_3$ and a random positive number rp_1 , and outputs the ciphertexts $C_R = rp_1(\tilde{M}_3RV_1, \dots, \tilde{M}_3RV_{2w})$.

$$RV_j = \begin{cases} (l_k h_k, l_k, h_k, 1)^T (j = 2k - 1) \\ (r_{k,1}, r_{k,2}, r_{k,3}, r_{k,4})^T (j = 2k) \end{cases} \tag{18}$$

(ii) As depicted in (19), the ZA inserts w random 4-d vectors into the w attribute vectors (corresponds to a w -d data point) included in a leaf node, where all random numbers $r'_k \ll d_k$, ($k \in [1, w]$). Later, M 1s are added to each vector $DV_i (i \in [1, 2w])$, then each DV_i is obfuscated by the $\Omega(\cdot)$ and K_Ω . Each obfuscated vector is encrypted with the key $\tilde{M}_3$ and a random positive number rp_2 , then the

ciphertexts $C_D = rp_2(\tilde{M}_3 DV_1, \dots, \tilde{M}_3 DV_{2w})$ are output by the ZA.

$$DV_i = \begin{cases} (d_k^3, d_k^2, d_k, 1)^T & (i = 2k - 1) \\ (r'_{k,1}, r'_{k,2}, r'_{k,3}, r'_{k,4})^T & (i = 2k) \end{cases} \quad (19)$$

Finally, the secure log index I containing the DAS is uploaded to the LAC platform together with the encrypted log files $\{E(log), E(U_i)\}$ for storage, and each index is identified by the GeoHash value of the aerial zone it belongs to.

C. GenToken

The user first determines the location query range L_Q and data query range MRQ , then executes this algorithm to generate the location query token TK_Q^L and log query token $\{TK_Q^D, TK_Q^R\}$ as follows.

- **Stage 1: Location Query Token Generation**

The user provides two query points $A(a_x, a_y, a_z)$ and $B(b_x, b_y, b_z)$ in one aerial zone, and takes the sphere constructed with the diameter of the line between the two points as the location query range. According to CMLD, the user first generates the plaintext of the location query vector Q_L with two points A, B , and adds N random numbers to Q_L to acquire Q'_L , where $\sum_{i=1}^N n_i = 0$, as shown in (20). Then the user obfuscates vector Q'_L as Q''_L with the $\Omega(\cdot)$ and K_Ω , and splits Q''_L to obtain two $(12 + N)$ -d vectors using the split vector S , as shown in (21).

$$\begin{aligned} Q_L &= (a_x b_x, -a_x, -b_x, 1, a_y b_y, -a_y, -b_y, 1, a_z b_z, \\ &\quad -a_z, -b_z, 1) \\ &\Downarrow \\ Q'_L &= (a_x b_x, -a_x, -b_x, 1, a_y b_y, -a_y, -b_y, 1, a_z b_z, \\ &\quad -a_z, -b_z, 1, \underbrace{n_1, \dots, n_N}_N) \\ &\Downarrow \\ Q''_L &= (\underbrace{-a_x, 1, a_y b_y, n_1, \dots, n_N, -b_x, 1, -b_z}_{12+N}) \quad (20) \\ &\begin{cases} Q_L^1[i] + Q_L^2[i] = Q''_L[i](S[i] = 0, i \in [1, 12 + N]) \\ Q_L^1[i] = Q_L^2[i] = Q''_L[i](S[i] = 1, i \in [1, 12 + N]) \end{cases} \quad (21) \end{aligned}$$

Later, the user generates the location query token $TK_Q^L = \{M_1 Q_L^1, M_2 Q_L^2\}$ with two keys M_1, M_2 .

- **Stage 2: Log Query Token Generation**

The user presents one data range query $MRQ = \{R_1, \dots, R_w\}$, where $R_k = [x_k^l, x_k^r](k \in [1, w])$. Based on CMLF, the user first generates the plaintexts of two-kind data query vectors DP_i and DR_i with MRQ . Then the user executes the below generation operations. (i) As shown in (22), the user inserts w random 4-d vectors into the w range query vectors $DR_j(j \in [1, w])$, where all

random numbers $r_k^R \ll x_{k,l}$ and $r_k^R \ll x_{k,r}$, ($k \in [1, w]$). Later, M random numbers m_i are added to each vector $DR_j(j \in [1, 2w])$, where $\sum_{i=1}^M m_i = 0$ and each DR_j is obfuscated by the $\Omega(\cdot)$ and K_Ω . The user encrypts each obfuscated vector with the key M_3 and a random positive number rp'_1 , then outputs the range query token $TK_Q^R = rp'_1(DR_1 M_3, \dots, DR_{2w} M_3)$.

$$DR_j = \begin{cases} \begin{pmatrix} x_k^r + \delta \\ -(x_k^r)^2 - x_k^r \delta \\ -x_k^l x_k^r - x_k^l \delta \\ x_k^l x_k^r (x_k^r + \delta) \end{pmatrix} & (j = 2k - 1) \\ \begin{pmatrix} r_{k,1}^R, r_{k,2}^R, r_{k,3}^R, r_{k,4}^R \end{pmatrix}^T & (j = 2k) \end{cases} \quad (22)$$

(ii) As shown in (23), the user inserts w random 4-d vectors into the w point query vectors $DP_i(i \in [1, w])$, where all random numbers $r_k^D \ll x_{k,l}$ and $r_k^D \ll x_{k,r}$, ($k \in [1, w]$). Later, M random numbers m_i are added to each vector $DP_i(i \in [1, 2w])$, where $\sum_{i=1}^M m_i = 0$ and each DP_i is obfuscated by the $\Omega(\cdot)$ and K_Ω . The user encrypts each obfuscated vector with the key M_3 and a random positive number rp'_2 , then outputs the point query token $TK_Q^D = rp'_2(DP_1 M_3, \dots, DP_{2w} M_3)$.

$$DP_i = \begin{cases} \begin{pmatrix} 1 \\ -x_k^l - x_k^r + \delta \\ x_k^l x_k^r - x_k^l \delta - x_k^r \delta \\ x_k^l x_k^r \delta \end{pmatrix} & (i = 2k - 1) \\ \begin{pmatrix} r_{k,1}^D, r_{k,2}^D, r_{k,3}^D, r_{k,4}^D \end{pmatrix}^T & (i = 2k) \end{cases} \quad (23)$$

Finally, the user transfers the location query token to the ZA, and transfers the log query tokens along with the GeoHash of the query aerial zone to the LAC platform.

D. Search

Once both the LAC platform and ZS receive the query tokens, they perform the following retrieval operations. (i) The ZS calculates the inner product between the location token TK_Q^L and the location data of UAVs. As demonstrated in (24), if $C_L \cdot TK_Q^L \leq 0$, the UAV U_i located in the location query L_Q ; Otherwise, U_i is outside the L_Q .

$$\begin{aligned} C_L \cdot TK_Q^L &= \{L_1 \cdot \tilde{M}_1, L_2 \cdot \tilde{M}_2\} \cdot \{M_1 \cdot Q_L^1, M_2 \cdot Q_L^2\} \\ &= |\det(M_1)| L_1 \cdot Q_L^1 + |\det(M_2)| L_2 \cdot Q_L^2 = L'' \cdot Q_L \\ &= \begin{pmatrix} a_x b_x - a_x l_x - b_x l_x + l_x^2 + a_y b_y - a_y l_y - b_y l_y \\ + l_y^2 + a_z b_z - a_z l_z - b_z l_z + l_z^2 + \sum_{i=1}^N n_i \end{pmatrix} \\ &= \vec{LA} \cdot \vec{LB} \quad (24) \end{aligned}$$

(ii) The LAC platform follows the G-tree retrieval. Starting from the root, it first calculates the inner product between the range query token TK_Q^R and the data range ciphertexts C_R starting from the root. If $TK_Q^R \cdot C_R \leq 0$, the LAC platform continues to traverse down; Otherwise, it abandons this non-leaf node. After the traversal reaches the leaf nodes, the LAC platform calculates the inner product between the point query token TK_Q^D and the data point ciphertexts C_D . If $TK_Q^D \cdot C_D \leq 0$, the data point falls within the query range; Otherwise, it is outside the range. Equations (25) and (26) demonstrate the above two calculation processes, respectively.

$$\begin{aligned}
& TK_Q^R \cdot C_R \\
&= \begin{cases} rp_1 rp'_1 |\det(M_3)| \langle DR_j \cdot RV_j \rangle (j = 2k - 1) \\ rp_1 rp'_1 |\det(M_3)| \sum_{i=1}^4 r_{k,i} r_{k,i}^R (j = 2k) \end{cases} \\
&= (rp_1 rp'_1 |\det(M_3)|) \{ \langle DR_j \cdot RV_j \rangle \}_{j=2k-1}^{k \in [1,w]} \\
&\quad + (rp_1 rp'_1 |\det(M_3)|) \left\{ \sum_{i=1}^4 r_{k,i} r_{k,i}^R \right\}_{j=2k}^{k \in [1,w]} \\
&\approx (rp_1 rp'_1 |\det(M_3)|) \{ \langle DR_j \cdot RV_j \rangle \}_{j=2k-1}^{k \in [1,w]} \quad (25)
\end{aligned}$$

$$\begin{aligned}
& TK_Q^D \cdot C_D \\
&= \begin{cases} rp_2 rp'_2 |\det(M_3)| \langle DP_i \cdot DV_i \rangle (i = 2k - 1) \\ rp_2 rp'_2 |\det(M_3)| \sum_{j=1}^4 r'_{k,j} r_{k,j}^D (i = 2k) \end{cases} \\
&= (rp_2 rp'_2 |\det(M_3)|) \{ \langle DP_i \cdot DV_i \rangle \}_{i=2k-1}^{k \in [1,w]} \\
&\quad + (rp_2 rp'_2 |\det(M_3)|) \left\{ \sum_{j=1}^4 r'_{k,j} r_{k,j}^D \right\}_{i=2k}^{k \in [1,w]} \\
&\approx (rp_2 rp'_2 |\det(M_3)|) \{ \langle DP_i \cdot DV_i \rangle \}_{i=2k-1}^{k \in [1,w]} \quad (26)
\end{aligned}$$

Finally, both the LAC platform and ZS return the query results $Result_L$ and $Result_Q$ to the user, where $Result_Q$ contains not only the ciphertexts of qualified log files, but also the AVI and the signatures of the reached leaf nodes during retrieval.

Remark 1: As we designed in *GenIndex* and *GenToken*, there exists $r_k \ll x_{k,l}, r_k \ll x_{k,r}, r'_k \ll d_k$ and $r_k^D, r_k^R \ll x_{k,l}, r_k^D, r_k^R \ll x_{k,r}$. Hence, the random numbers only serve as perturbations to preserve sensitive information, but cannot have an effect on the positive or negative of the inner products $TK_Q^R \cdot C_R$ and $TK_Q^D \cdot C_D$, i.e., the probability of returning the data point is negligible if DP is not inside the data range.

E. Verify

Since data completeness verification is only aimed at data queries but not location queries, the AVI formation and completeness verification process are illustrated with the assistance of Fig. 6. We also describe the result decryption with a verification flag.

AVI Formation: We assume that ID_4 and ID_5 are the query results, the AVI returned by the LAC platform consists

of the signatures $Sig(Hash(NL_3)), Sig(Hash(NL_6))$ of the non-leaf nodes NL_3, NL_6 , the signatures $Sig(Hash(ID_3)), Sig(Hash(ID_6))$ of the leaf nodes ID_3, ID_6 and the signature of the root $Sig(Hash(Root))$.

Completeness Verification: The user first decrypts each signature in AVI and $Result_Q$ with the RSA public key pk to obtain the Hash set using:

$$\{Hash(NL_3), Hash(NL_6), Hash(ID_3), Hash(ID_6)\} \quad (27)$$

Combined with the $Hash(ID_3), Hash(ID_6)$ of leaf nodes ID_3, ID_6 , the user reconstruct the $Hash'(Root)$ from the bottom as described in (28). If $Hash'(Root) = Hash(Root)$, the query results are complete; Otherwise, the query results are incomplete.

$$\begin{aligned}
& Hash(Hash(NL_3) || Hash(ID_3) || Hash(ID_4) || \\
& Hash(ID_5) || Hash(ID_6) || Hash(NL_6)) \\
&= Hash(Hash(NL_1) || Hash(NL_2)) \\
&= Hash'(Root) \quad (28)
\end{aligned}$$

Result Decryption: Based on Definition 5, if the completeness verification passes, indicated by the flag $b = 1$, the users decrypt the ciphertexts D_R with the symmetric key k and obtain the plaintext result P_R . Otherwise, the users reject to decrypt the ciphertexts D_R .

VII. SECURITY ANALYSIS

In this section, we prove the semantic security of LSPSS for Chosen Plaintext Attack (CPA), single-dimensional privacy, and query unlinkability during search based on the Known Background Model.

A. Security Under Known Background Model

Lemma 1: The LSPSS satisfies the semantic security if the encrypted data are indistinguishable under Chosen Plaintext Attack (IND-CPA).

Proof: We only require to prove that the probability of the probabilistic polynomial-time (PPT) adversary breaking the encrypted data of the LSPSS is negligible. Suppose the challenger $\mathcal{C}$ performs *GenKey*(1^λ) to provide an LSPSS encryption system $K = \{K_1, K_2\}$ and transfers K_2 to a PPT adversary $\mathcal{A}$. Then $\mathcal{A}$ generates and uploads two data records d_0, d_1 to the $\mathcal{C}$. The $\mathcal{C}$ randomly selects $b \in \{0, 1\}$, encrypts d_b using K_1 , and returns the ciphertext to $\mathcal{A}$. Later the $\mathcal{A}$ presents a guess b' of b . Due to the perturbation and permutation of the encryption, random numbers are employed each time, so converting one plaintext into diverse ciphertexts based on the identical key. Additionally, since the matrix keys are randomly generated, it is hard to be broken. In conclusion, it is obvious to deduce that $\mathcal{A}$ cannot guess the correct b with a probability high than $1/2$. Hence the advantage in this game is

$$Adv_{LSPSS, \mathcal{A}}^{CPA}(\lambda) = \left| \Pr[b = b'] - \frac{1}{2} \right| < \text{negl}(\lambda)$$

Thus the LSPSS is proved to be semantically secure. $\square$

Based on *Security Definition*, given two histories H and H' with the same trace Tr , if the server with the distribution of queries cannot distinguish which view of them is generated by the simulator $\mathcal{S}$, they cannot learn extra information beyond what can be leaked. Thus the LSPSS is secure under the known background model.

Theorem 1: LSPSS is secure against semi-honest adversaries under the known background model.

Proof: The simulator $\mathcal{S}$ can simulate a view V' indistinguishable from the view $V(H) = \{D^*, L^*, I_D^*, I_L^*, TK(D), TK(L)\}$ of the server as follows.

- $\mathcal{S}$ randomly selects a $d_i' \in \{0, 1\}^{|d_i^*|}$, $d_i^* \in D^*$ ($i \in [1, |D^*|]$) and a $l_j' \in \{0, 1\}^{|l_j^*|}$, $l_j^* \in L^*$ ($j \in [1, |L^*|]$), then generates $D' = \{d_i'\} (i \in [1, |D^*|])$ and $L' = \{l_j'\} (j \in [1, |L^*|])$.
- $\mathcal{S}$ randomly selects two invertible matrices $M_1', M_2' \in \mathbb{R}^{4 \times 4}$, $M_3' \in \mathbb{R}^{12 \times 12}$ and forms $K' = \{M_1', M_2', M_3'\}$. Also, $\mathcal{S}$ generates a permutation function $\Omega'(\cdot)$ and the key K'_Ω .
- $\mathcal{S}$ generates one vector of $2w$ elements for each $v_i' \in I_D'(D')$, $i \in [1, |I_D^*|]$ and one vector $u_j' \in I_L'(L')$, $j \in [1, |I_L^*|]$ as the data index and location index, respectively. Then $\mathcal{S}$ performs as follows.
 - (i) For each element of v_i' , $\mathcal{S}$ replaces it with one 4-d vector $(r_{i,1}, \dots, r_{i,4})^T$, where $r_{i,k}$ ($i \in [1, n]$) is a random number. For each element of u_j' , $\mathcal{S}$ also replaces it with one 12-d vector $(r'_{j,1}, \dots, r'_{j,12})^T$, where $r'_{j,k}$ ($j \in [1, m]$) also be a random number. Later $\mathcal{S}$ obfuscates each v_i' and u_j' with the $\Omega'(\cdot)$ and K'_Ω .
 - (ii) $\mathcal{S}$ encrypts each v_i' and u_j' with the M_2' and M_3' , respectively. Then $\mathcal{S}$ obtains $I_{D'}' = \{Enc_{K'}(v_i'), i \in [1, |I_D^*|]\}$ and $I_{L'}' = \{Enc_{K'}(u_j'), j \in [1, |I_L^*|]\}$.
- $\mathcal{S}$ builds the query token $TK(D')$ and $TK(L')$ as follows.
 - (i) For each $qd_i' \in Q'(D') (i \in [1, \tau])$, $\mathcal{S}$ generates one vector qv_i' of $2w$ elements and replaces each element with one 4-d vector $(dr_{i,1}, \dots, dr_{i,4})^T$, where $dr_{i,k}$ ($i \in [1, n]$) is a random number. For each element $ql_j' \in Q'(L') (j \in [1, \omega])$, $\mathcal{S}$ generates one vector qu_j' and replaces it with one 12-d vector $(lr'_{j,1}, \dots, lr'_{j,12})^T$, where $lr_{j,k}$ ($j \in [1, m]$) also be a random number. Then $\mathcal{S}$ obfuscates each qv_i' and qu_j' with the $\Omega'(\cdot)$ and K'_Ω .
 - (ii) $\mathcal{S}$ respectively encrypts each qv_i' and qu_j' using M_1' and $M_3'^{-1}$ to generate the query tokens for each qd_i' and ql_j' . Thus $\mathcal{S}$ acquires $TK(D')$ and $TK(L')$.
- $\mathcal{S}$ outputs the view $V' = \{D', L', I_{D'}', I_{L'}', TK(D'), TK(L')\}$.

It is obvious to prove the correctness of the above construction with querying $TK(D')$, $TK(L')$ on $I_{D'}'$, $I_{L'}'$. The trace generated by the indexes $TK(D')$, $TK(L')$ and tokens $I_{D'}'$, $I_{L'}'$ is the same as the trace of the server. We affirm that no PPT adversary can distinguish the view V' from $V(H)$.

Specifically, since the semantic security of LSPSS, no PPT adversary can distinguish the D^* , L^* from D' , L' . Also, due to the indistinguishability of random perturbation and permutation of the encryption, the PPT adversary with queries and

probability pairs cannot distinguish which tokens are generated by the same queries. Thus the PPT adversary cannot employ the distributions of ciphertexts and encrypted tokens to infer sensitive information. $\square$

B. Single-Dimensional Privacy

Theorem 2: LSPSS can preserve single-dimensional privacy during retrievals.

Proof: We employ two-kind perturbation-based matrix encryption for location range query and data range query, respectively. Based on the query mechanism of two-kind encryption, the servers are only able to know the intersection results by inner product calculation. Moreover, with the permutation and perturbation of matrix encryption, the servers cannot distinguish the obfuscated leaf nodes or non-leaf nodes in the secure data index. Thus the servers cannot deduce the information on which records satisfy each single dimension of the query, i.e., the LSPSS can preserve single-dimensional privacy during retrievals. $\square$

C. Query Unlinkability

Theorem 3: LSPSS can achieve query unlinkability during query token generation.

Proof: (i) Suppose that the user presents one location range query L_Q and generates one original query vector Q_L . The user generates η location query tokens with the identical query Q_L . Based on the token generation algorithm *GenToken*, the user first extends η vectors Q_L to $(Q_L')_i (i \in [1, \eta])$ with N random numbers n_i , where $\sum_{i=1}^N n_i = 0$. Then, after splitting the $(Q_L')_i$ using the split vector S , the user exploits the keys M_1 and M_2 to encrypt $\{(Q_L')_i^1, (Q_L')_i^2\} (i \in [1, \eta])$ and output the tokens $\{TK_k(Q_L)\}_{k=1}^\eta$. Note that N random numbers n_i inserted into each Q_L just need to satisfy $\sum_{i=1}^N n_i = 0$, thus there exists infinite combinations, i.e., none of the η location tokens are the same. According to Definition 3, the location range query in LSPSS realizes query unlinkability.

(ii) Similarly, suppose that the user presents one data range query MRQ and generates two original query vector sets $Q_R = \{DR_j\} (j \in [1, w])$ and $Q_D = \{DP_i\} (i \in [1, w])$. The user generates η data range tokens with the identical query MRQ . Based on *GenToken*, the user first inserts w 4-d random vectors into each vector set $Q_R^k (k \in [1, \eta])$ and $Q_D^k (k \in [1, \eta])$, respectively. Then the user extends each vector DR_j and DP_i in η sets Q_R^k and Q_D^k with M random numbers m_i , where $\sum_{i=1}^M m_i = 0$. Later, the user employs the key M_3 to encrypt $Q_R^k = \{DR_j, (j \in [1, 2w])\}_{k=1}^\eta$ and $Q_D^k = \{DP_i, (i \in [1, 2w])\}_{k=1}^\eta$ and output the tokens $\{TK_k(Q_R)\}_{k=1}^\eta$ and $\{TK_k(Q_D)\}_{k=1}^\eta$. Note that M random numbers m_i inserted into each Q_R and Q_D just need to satisfy $\sum_{i=1}^M m_i = 0$. This results in infinite combinations, i.e., none of the η data tokens are the same. According to Definition 3, the data range query in LSPSS achieves query unlinkability.

In conclusion, LSPSS can achieve query unlinkability during query token generation. $\square$

TABLE I
SCHEMES COMPARISON

Scheme	Asymptotic Query Complexity	Basic Operation	Query Unlinkability	Single-dimensional Privacy	Result Verification
Maple [17]	$O(\log x \cdot d)$	Bilinear Pairing	×	✓	×
WangR [24]	$O(\log x \cdot d)$	Multiplication over R	×	×	×
PRQ [28]	$O(\log x \cdot d \cdot N^2)$	Multiplication over R	✓	✓	×
LSPSS	$O(\log x \cdot d)$	Multiplication over R	✓	✓	✓

VIII. PERFORMANCE EVALUATION

To evaluate the comprehensive performance, we first compare our LSPSS with other similar schemes from the view of index construction and querying. Furthermore, we discuss the performance of our LSPSS in index construction, query token generation, querying, and result verification.

Experimental Settings: We implement our LSPSS with Python (Python 3.7) on a PC (AMD Ryzen 5 4600 U Radeon Graphics 2.10 Hz, Windows 10). Additionally, we exploited a UAV environmental monitoring log dataset called *UAVs Detection* from the Kaggle.² The attributes we extracted include the Location (i.e., altitude, latitude, and longitude), Temperature, Humidity, Concentration, Air Velocity, Light and Radiation Intensity.

Parameter Settings: We set the experimental parameter as the dataset size 5000, 10000, 15000, 20000, 25000, the attribute dimension from 2 to 9 (i.e., three dimensions for location and six dimensions for log files), and the number of UAVs 500, 1000, 1500, 2000, 2500 in each aerial zone. The keys' bit-size of RSA and AES are fixed as 1024 and 128. For the objectivity of experiments, we randomly choose the query ranges and acquire the averaging results of repeated experiments.

A. Comparison With Prior Arts

According to *Related Works*, data privacy preservation in aerial computing is mainly implemented with the assistance of blockchain and federated learning (FL) techniques. To compare the performance of LSPSS with other cryptography-based schemes, we first present a comprehensive comparison of LSPSS and other privacy-preserving MRQ schemes in Table I. Later, we perform the experiments of Maple, PRQ and LSPSS in index construction and querying. Note that the bilinear pairing operations in Maple are implemented with *PyPBC* library (512 bits security parameter), we also achieve the SHA-256 with *PySHA3* library supported by Python.

1) *Index Construction:* Concretely, the experimental results of index construction are shown in Fig. 7(a) and (b) from the perspectives of time and storage consumption. The attribute dimension and the size of matrix keys are fixed to $w = 2$ and (16×16) respectively, as well as the dataset size $|D|$ varies from 5000 to 25000. As we observed, the time and storage consumption both show a linear increasing trend as $|D|$ increases. By comparison, Maple applies public-key cryptography called hidden vector encryption (HVE) to achieve selective security,

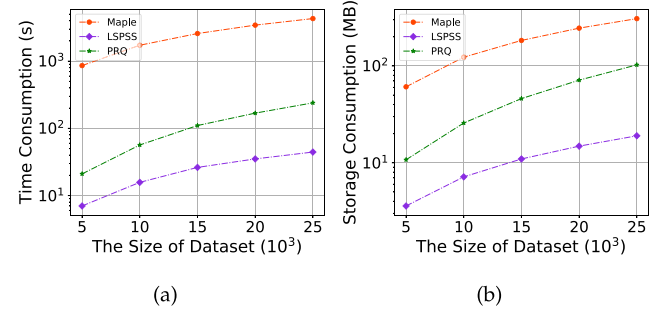


Fig. 7. (a) The time consumption of index construction for three schemes with diverse datasets. (b) The storage consumption of index construction for three schemes with diverse datasets.

thus LSPSS is on average about 120 and 16 times smaller than Maple in terms of time and storage consumption, respectively. And compared with PRQ, the time consumption of PRQ is also larger than that of LSPSS because of the larger coefficient of the time complexity.

2) *Query:* In comparison with prior arts of querying, we perform the experiments from the perspective of dataset size and dimension number, respectively. Moreover, we also experimentally compare the query precision and recall rate of the three schemes under the same conditions.

Querying: Specifically, Fig. 8(a) and (b) depict the time consumption of querying with diverse schemes. The size of matrix keys is fixed to (16×16) -dimensions for PRQ and LSPSS in these experiments. We set the attribute dimension as $w = 2$, as well as the dataset size $|D|$ varies from 5000 to 25000 in Fig. 8(a). Moreover, the dataset size is set as $|D| = 25000$ and the attribute dimension w varies from 2 to 6 in Fig. 8(b). Intuitively, the time consumption of querying linearly increases with the sizes of the dataset and the attribute dimension. Maple is on average about 900 and 850 times larger than LSPSS in terms of the datasets and attribute dimensions, respectively. Compared with PRQ, the time consumption of LSPSS is slightly smaller than that of PRQ, not only because of the smaller coefficient query time complexity, but also due to the setting of the variable-length partition number in G-tree, which reduces the effect of data amount and distribution on the query.

Precision and Recall: Generally speaking, precision is defined as the proportion of true results in the whole query results, and recall is defined as the proportion of true query results in the whole true results. The calculation of precision and recall are depicted as (29) and (30), where TP and FP represent the number of true and false results, and FN represents the number

²<https://www.kaggle.com>

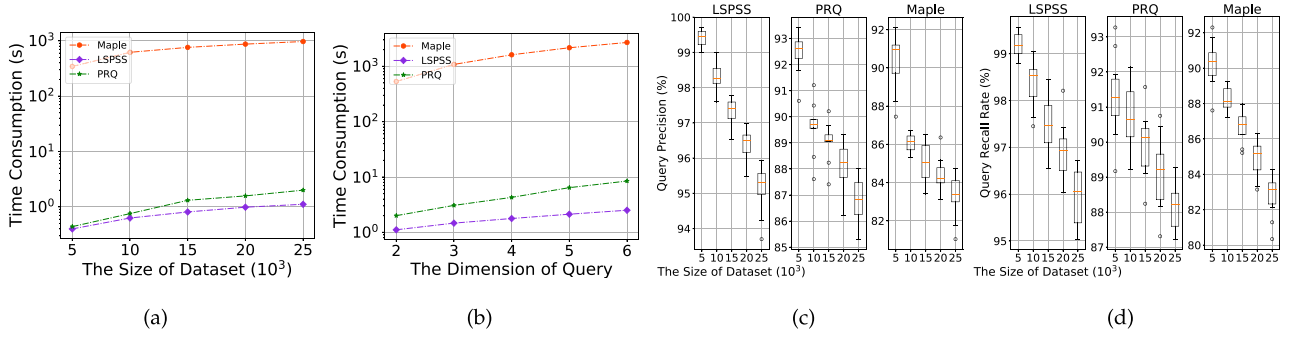


Fig. 8. (a) The time consumption of querying for three schemes with diverse datasets. (b) The time consumption of querying for three schemes with diverse attributes. (c) The comparison of query precision for diverse schemes with diverse datasets. (d) The comparison of query recall rate for three schemes with diverse datasets.

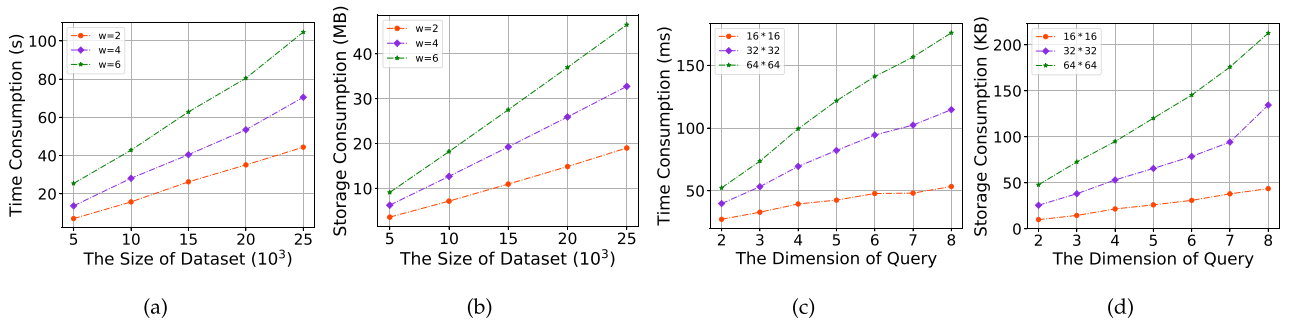


Fig. 9. (a) The time consumption of index construction with diverse datasets and attribute numbers. (b) The storage consumption of index construction with diverse datasets and attribute numbers. (c) The time consumption of query token generation with matrix key and attribute numbers. (d) The storage consumption of query token generation with matrix key and attribute numbers.

of true results that have been retrieved.

$$Precision = \frac{TP}{TP + FP} \quad (29)$$

$$Recall = \frac{TP}{TP + FN} \quad (30)$$

Concretely, the experimental results of query precision and recall varying $|D|$ from 5000 to 25000, as well as the sizes of matrix keys and attribute dimensions are fixed to (16×16) -dimensions and $w = 8$, respectively. We exploit the same query Q to conduct 50 tests on the three schemes, then draw two box plots for query precision and recall in Fig. 8(c) and (d). The indexes of PRQ and Maple are built with R-tree, while that of LSPSS is built with G-tree. Compared to the R-tree, the sibling nodes in the G-tree are not allowed to overlap, thus our LSPSS achieves higher precision and recall than the other two schemes. Also compared with PRQ and Maple, the outliers generated by LSPSS are less than that of the other two schemes during the query, which indicates that our LSPSS has higher query stability.

B. Self-Performance Evaluation

1) *Index Construction*: As the design of LSPSS, dataset size, keys size and attribute dimensions all affect the index construction. Fig. 9(a) and (b) describe the time and storage consumption of index construction with diverse datasets and attribute dimensions. We set the size of matrix keys M_1 , M_2 and M_3 as (16×16) -dimensions. Obviously, the construction time and storage consumption increase linearly with the dataset

size $|D|$ and attribute dimensions w , since our LSPSS encrypts two-kind data features with the matrix encryption methods we proposed. In other words, LSPSS improves query efficiency and security at the expense of time and storage consumption.

2) *Token Generation*: The dominant factors in query token generation are the size of matrix keys and attribute dimensions, thus we plot the time and storage consumption of token generation varying with the above two factors in Fig. 9(c) and (d). As shown in the two figures, we set the attribute dimension $w \in [2, 8]$ and three key sizes. Clearly, the generation time and storage consumption increase linearly with the attribute dimensions w and key size. The larger the key is set, the more secure the token is, but the resulting consumption becomes larger, so we should seek a trade-off between security and efficiency.

3) *Query*: The main factors affecting the query include the size of the dataset and attribute dimensions. Fig. 10(a) demonstrates the time consumption varies with the diverse sizes of the attribute dimensions $w \in \{2, 4, 6\}$ and dataset $|D| \in \{5 + 0.5, 10 + 1, 15 + 1.5, 20 + 2, 25 + 2.5\} \times 10^3$, where $\{0.5, 1, 1.5, 2, 2.5\} \times 10^3$ donates as the number of UAVs in each aerial zone, that is, each UAV generates one location record and collects ten log files. We also fix the size of three matrix keys to (16×16) -dimensions. From the perspective of attribute dimensions, the time consumption of querying linearly increases with the w in each dataset. As for the dataset size, the location and log files are indexed by the inverted and tree structures, respectively. The location query on the inverted index dominates the consumption when the dimension $w \leq 4$ and the dataset $|D| \leq 15 + 1.5$, thus the time

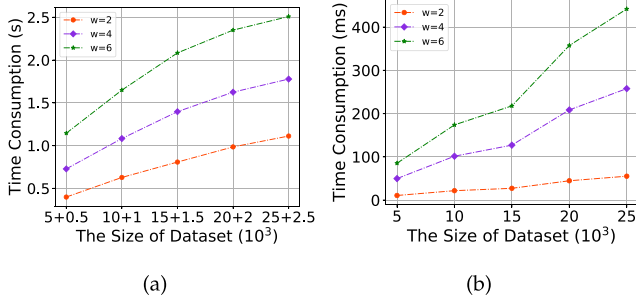


Fig. 10. (a) The time consumption of querying with diverse datasets and attribute numbers. (b) The time consumption of result verification with diverse datasets and attribute numbers.

consumption shows a linear growth. However, the data query on the tree index dominates the consumption when the $w > 4$ and $|D| > 15 + 1.5$, so the time consumption shows a sub-linear growth and continues.

4) *Result Verification*: Similarly, the time consumption of result verification is affected by the size of the dataset and attribute dimensions. Fig. 10(b) shows the time consumption varies with the diverse sizes of the attribute dimensions $w \in \{2, 4, 6\}$ and dataset $|D| \in \{5, 10, 15, 20, 25\} \times 10^3$, as well as the size of matrix keys is fixed to (16×16) -dimensions. Evidently, the time consumption increases linearly with the dataset size $|D|$. Note that the variation in dataset size has a greater effect on the verification consumption when the attribute dimension w is larger.

IX. CONCLUSION

In this article, we have proposed a lightweight and secure private data storage and sharing scheme to support MRQ and query result verification over encrypted data in the LAC platform of aerial computing. The security of our approach is proven under exactly-defined leakages. We first design two conversion methods to process two kinds of multi-dimensional data. Based on this, we build a secure data index and a DAS with G-tree which enables efficient ciphertext retrieval and query result verification. Practically, via experiments on real-world datasets, we show that our approach is practical for a large-scale dataset and can achieve up to $900\times$ and $2\times$ query time improvement compared to the prior arts [17], [28], respectively.

In our future work, we consider further enhancing the security of the solution. This will include hiding the access pattern and search pattern during the query, as well as reducing the time and storage consumption required for result verification.

REFERENCES

- [1] W. Li, Z. Su, R. Li, K. Zhang, and Y. Wang, "Blockchain-based data security for artificial intelligence applications in 6G networks," *IEEE Netw.*, vol. 34, no. 6, pp. 31–37, Nov/Dec. 2020.
- [2] W. Sun, L. Wang, P. Wang, and Y. Zhang, "Collaborative blockchain for space-air-ground integrated networks," *IEEE Wireless Commun.*, vol. 27, no. 6, pp. 82–89, Dec. 2020.
- [3] K. Lei, Q. Zhang, J. Lou, B. Bai, and K. Xu, "Securing ICN-based UAV ad hoc networks with blockchain," *IEEE Commun. Mag.*, vol. 57, no. 6, pp. 26–32, Jun. 2019.
- [4] T. Liu, H. Guo, C. Danilov, and K. Nahrstedt, "A privacy-preserving data collection and processing framework for third-party UAV services," in *Proc. IEEE 19th Int. Conf. Trust Secur. Privacy Comput. Commun.*, 2020, pp. 683–690.

- [5] Y. Tian, J. Yuan, and H. Song, "Efficient privacy-preserving authentication framework for edge-assisted internet of drones," *J. Inf. Secur. Appl.*, vol. 48, 2019, Art. no. 102354.
- [6] Y. Wang, Z. Su, N. Zhang, and A. Benslimane, "Learning in the air: Secure federated learning for UAV-assisted crowdsensing," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1055–1069, Second Quarter, 2021.
- [7] Y. Lu et al., "Privacy-preserving logarithmic-time search on encrypted data in cloud," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2012.
- [8] A. Boldyreva, N. Chenette, Y. Lee, and A. O'Neill, "Order-preserving symmetric encryption," in *Proc. 28th Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, Springer, 2009, pp. 224–241.
- [9] A. Boldyreva, N. Chenette, and A. O'Neill, "Order-preserving encryption revisited: Improved security analysis and alternative solutions," in *Proc. 31st Annu. Cryptol. Conf.*, Springer, 2011, pp. 578–595.
- [10] L. Xiao and I.-L. Yen, "A note for the ideal order-preserving encryption object and generalized order-preserving encryption," *Cryptol. ePrint Arch.*, Paper 2012/350, 2012. [Online]. Available: <https://eprint.iacr.org/2012/350>
- [11] H. Hacigümüş, B. Iyer, C. Li, and S. Mehrotra, "Executing SQL over encrypted data in the database-service-provider model," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2002, pp. 216–227.
- [12] B. Hore, S. Mehrotra, M. Caim, and M. Kantarcioglu, "Secure multidimensional range queries over outsourced data," *VLDB J.*, vol. 21, pp. 333–358, 2012.
- [13] Y. Lee, "Secure ordered bucketization," *IEEE Trans. Dependable Secure Comput.*, vol. 11, no. 3, pp. 292–303, May/Jun. 2014.
- [14] H. G. Do and W. K. Ng, "Multidimensional range query on outsourced database with strong privacy guarantee," in *Proc. 14th Annu. Conf. Privacy Secur. Trust*, 2016, pp. 555–560.
- [15] E. Shi, J. Bethencourt, T. H. Chan, D. Song, and A. Perrig, "Multidimensional range query over encrypted data," in *Proc. IEEE Symp. Secur. Privacy*, 2007, pp. 350–364.
- [16] D. Boneh and B. Waters, "Conjunctive, subset, and range queries on encrypted data," in *Proc. 4th Theory Cryptogr. Conf.*, Springer, 2007, pp. 535–554.
- [17] B. Wang, Y. Hou, M. Li, H. Wang, and H. Li, "Maple: Scalable multidimensional range search over encrypted cloud data with tree-based index," in *Proc. 9th ACM Symp. Inf. Comput. Commun. Secur.*, 2014, pp. 111–122.
- [18] R. Xu, K. Morozov, Y. Yang, J. Zhou, and T. Takagi, "Privacy-preserving k-nearest neighbour query on outsourced database," in *Proc. 21st Australas. Conf. Inf. Secur. Privacy*, Springer, 2016, pp. 181–197.
- [19] Y. Zheng and R. Lu, "An efficient and privacy-preserving k-NN query scheme for healthcare data," in *Proc. IEEE Int. Conf. Internet Things IEEE Green Comput. Commun. IEEE Cyber Phys. Social Comput. Smart Data*, 2018, pp. 358–365.
- [20] Z. Xu, Y. Lin, V. K. A. Sandor, Z. Huang, and X. Liu, "A lightweight privacy and integrity preserving range query scheme for mobile cloud computing," *Comput. Secur.*, vol. 84, pp. 318–333, 2019.
- [21] Y. Zheng, R. Lu, Y. Guan, J. Shao, and H. Zhu, "Efficient and privacy-preserving similarity range query over encrypted time series data," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2501–2516, Jul./Aug. 2022.
- [22] Y. Guo, H. Xie, M. Wang, and X. Jia, "Privacy-preserving multi-range queries for secure data outsourcing services," *IEEE Trans. Cloud Comput.*, vol. 11, no. 3, pp. 2431–2444, Third Quarter 2023.
- [23] Y. Yang, S. Papadopoulos, D. Papadias, and G. Kollios, "Authenticated indexing for outsourced spatial databases," *VLDB J.*, vol. 18, pp. 631–648, 2009.
- [24] P. Wang and C. V. Ravishanker, "Secure and efficient range queries on outsourced databases using Rp-trees," in *Proc. IEEE 29th Int. Conf. Data Eng.*, 2013, pp. 314–325.
- [25] J. Chi, C. Hong, M. Zhang, and Z. Zhang, "Fast multi-dimensional range queries on encrypted cloud databases," in *Proc. 22nd Int. Conf. Database Syst. Adv. Appl.*, Springer, 2017, pp. 559–575.
- [26] B. Wang, M. Li, and H. Wang, "Geometric range search on encrypted spatial data," *IEEE Trans. Inf. Forensics Secur.*, vol. 11, no. 4, pp. 704–719, Apr. 2016.
- [27] W. Yang, Y. Geng, L. Li, X. Xie, and L. Huang, "Achieving secure and dynamic range queries over encrypted cloud data," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 1, pp. 107–121, Jan. 2022.
- [28] Y. Zheng, R. Lu, Y. Guan, J. Shao, and H. Zhu, "Towards practical and privacy-preserving multi-dimensional range query over cloud," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3478–3493, Sep./Oct. 2022.
- [29] Y. Zheng et al., "PMRQ: Achieving efficient and privacy-preserving multidimensional range query in eHealthcare," *IEEE Internet Things J.*, vol. 9, no. 18, pp. 17 468–17 479, Sep. 2022.

- [30] S. Wu, Q. Li, G. Li, D. Yuan, X. Yuan, and C. Wang, "ServeDB: Secure, verifiable, and efficient range queries on outsourced database," in *Proc. IEEE 35th Int. Conf. Data Eng.*, 2019, pp. 626–637.
- [31] L. Chen, N. Zhang, H.-M. Sun, C.-C. Chang, S. Yu, and K.-K. R. Choo, "Secure search for encrypted personal health records from big data NoSQL databases in cloud," *Computing*, vol. 102, pp. 1521–1545, 2020.
- [32] Z. Mei et al., "Executing multi-dimensional range query efficiently and flexibly over outsourced ciphertexts in the cloud," *Inf. Sci.*, vol. 432, pp. 79–96, 2018.
- [33] Y. Zhan, D. Shen, P. Duan, B. Zhang, Z. Hong, and B. Wang, "MDOPE: Efficient multi-dimensional data order preserving encryption scheme," *Inf. Sci.*, vol. 595, pp. 334–343, 2022.
- [34] A. Kumar, "G-Tree: A new data structure for organizing multidimensional data," *IEEE Trans. Knowl. Data Eng.*, vol. 6, no. 2, pp. 341–347, Apr. 1994.
- [35] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Proc. Conf. Theory Appl. Cryptographic Techn.*, Berlin, Germany: Springer-Verlag, 1987, pp. 369–378.
- [36] W. K. Wong, D. W.-L. Cheung, B. Kao, and N. Mamoulis, "Secure kNN computation on encrypted databases," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2009, pp. 139–152.



Haoyang Wang received the BS degree in software engineering from Northeastern University, Shenyang, China, in 2018, and the master's degree in computer technology from Xidian University, in 2021. He is currently working toward the PhD degree with the State Key Laboratory of Integrated Service Networks, Xidian University. His research interests include data privacy-preserving, searchable encryption, and Internet of Vehicles Security.



Kai Fan (Member, IEEE) received the BS, MS, and PhD degrees from Xidian University, P. R. China, in 2002, 2005, and 2007, respectively, in telecommunication engineering, cryptography and telecommunication and information system. He is working as a professor with the State Key Laboratory of Integrated Service Networks, Xidian University. He published more than 70 papers in journals and conferences. He received nine Chinese patents. He has managed five national research projects. His research interests include IoT security and information security.



Chong Yu (Graduate Student Member, IEEE) received the BSc degree in communication engineering and the MSc degree in communication and information system from Northeastern University, Shenyang, China, in 2015 and 2017, respectively. She is currently working toward the PhD degree with the Department of Electrical and Computer Engineering, University of Nebraska-Lincoln, USA, in 2023. She is currently with the Department of Computer Science, University of Cincinnati, OH, USA. Her research interests include artificial intelligence (AI), machine learning,

and cybersecurity with a broad range of applications including data analytics, edge-based AI, and Internet of Things.



Kuan Zhang (Member, IEEE) received the BS and MS degrees from Northeastern University, Shenyang, China, in 2009 and 2011, respectively, in communication engineering and computer applied technology, and the PhD degree from the University of Waterloo, Canada, in 2016, in electrical and computer engineering. He was a postdoctoral fellow from 2016–2017 with the University of Waterloo, Canada. He is working as an Assistant Professor with the Department of Electrical and Computer Engineering, University of Nebraska-Lincoln, USA. He has published more

than 50 papers in journals and conferences. He was the recipient of Best Paper Award in IEEE WCNC 2013 and Securecomm 2016. His research interests include cyber security, Big Data, and cloud/edge computing.



Fenghua Li (Member, IEEE) received the PhD degree from Xidian University. He is now a professor with the Chinese Academy of Sciences and Xi'an University, Xi'an China. His research interests include cyber and system security, information protection, and data security.



Hui Li (Member, IEEE) received the BS degree in radio electronics from Fudan University, and the MS and PhD degrees in telecommunications and information system from Xidian University, in 1993 and 1998, respectively. He is now a professor with Xidian University. His research interests include network and information security.



Yintang Yang (Senior Member, IEEE) received the PhD degree in semiconductor from Xidian University. He is now a professor with the Key Laboratory of Ministry of Education for Wide Band-Gap Semiconductor Materials and Devices, Xidian University, Xi'an China. His research interests include semiconductor materials and devices, network and information security.



Haojin Zhu (Fellow, IEEE) received the BSc degree from Wuhan University, China, in 2002, the MSc degree from Shanghai Jiao Tong University, China, in 2005, both in computer science, and the PhD degree in electrical and computer engineering from the University of Waterloo, Canada, in 2009. He is currently a professor with Computer Science Department, Shanghai Jiao Tong University. His current research interests include network security and privacy enhancing technologies. He published more than 70 international journal papers, including *IEEE Journal*

on Selected Areas in Communications, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Information Forensics and Security*, and 90 international conference papers, including IEEE S&P, ACM CCS, USENIX Security, NDSS, ACM MOBIKOM. He received a number of awards including: ACM CCS Best Paper Runner-Ups Award (2021), IEEE TCSC Award for Excellence in Scalable Computing (Middle Career Researcher, 2020), IEEE ComSoc Asia-Pacific Outstanding Young Researcher Award (2014), Top 100 Most Cited Chinese Papers Published in International Journals (2014), distinguished member of the IEEE INFOCOM Technical Program Committee (2015, 2020), best paper awards of IEEE ICC (2007) and Chinacom (2008), WASA Best Paper Runner-up Award (2017). He is serving the editorial board of *IEEE Transactions on Wireless Communications* and program committees for top conferences such as USENIX Security, ACM CCS, NDSS, and IEEE INFOCOM.